

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration: 1 Hour
Total Marks:50

Annexure A

Scenario Based Assignment

Code of Conduct:

I _____ R.S.Adithiya(192512053)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: R.S.Adithiya

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

- i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty
- ii. Explain how A* would handle the same situation, including how it balances cost and heuristic
- iii. Analyze the impact of heuristic accuracy on both algorithms
- iv. Discuss how dynamic changes (blocked paths) affect search performance
- v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

ANSWER:

SCENARIO: AI-POWERED DRONES FOR EMERGENCY MEDICINE DELIVERY IN FLOOD-AFFECTED REGIONS

INTRODUCTION

A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic because roads and paths may suddenly become inaccessible, while weather conditions such as heavy rain and strong winds can change the travel cost and risk. The drone has access to a heuristic estimate based on straight-line distance, partial real-time information about blocked paths, and variable movement costs depending on terrain and weather conditions. Since emergency medicine delivery is time-critical, the drone must select routes quickly while considering both delay and safety.

I. BEHAVIOR OF GREEDY BEST-FIRST SEARCH

Greedy Best-First Search (GBFS) selects the next state based only on the heuristic value $h(n)$. In this scenario, the heuristic can be the straight-line distance between the drone's current location and the medicine delivery destination.

The evaluation function is:

$$f(n) = h(n)$$

where $h(n)$ represents the estimated distance from the current location to the destination.

The drone will always prefer the path that appears closest to the destination according to the heuristic. For example, if two possible routes are available, the drone will select the route that has the smaller estimated distance to the destination, even if that route has difficult terrain, bad weather, or higher risk.

STRENGTHS OF GREEDY BEST-FIRST SEARCH:

1. It is generally fast because it focuses directly on the destination.
2. It can quickly find a route in large search spaces.
3. It is useful when an immediate decision is more important than finding the globally optimal route.
4. It can be effective when the heuristic provides a good estimate of the remaining distance.

WEAKNESSES UNDER UNCERTAINTY:

1. It ignores the actual cost already spent in reaching the current location.
2. It may select a route with high terrain or weather costs.
3. It is not guaranteed to find the optimal route.
4. If the shortest-looking path suddenly becomes blocked, the drone may need to perform additional searching.
5. An inaccurate heuristic can cause the algorithm to make poor decisions.
6. It may select a dangerous route simply because it appears geographically closer.

Therefore, Greedy Best-First Search is fast and suitable for quick decisions, but it may not be reliable enough when safety, weather, and variable travel costs are important.

II. BEHAVIOR OF A* SEARCH

A* Search combines the actual cost of reaching a state with the estimated cost of reaching the destination. It uses the following evaluation function:

$$f(n) = g(n) + h(n)$$

where:

$g(n)$ = Actual cost from the starting point to the current node

$h(n)$ = Estimated cost from the current node to the destination

$f(n)$ = Estimated total cost of the complete route

In this scenario, $g(n)$ can include travel time, terrain difficulty, weather conditions, battery consumption, and risk. The heuristic $h(n)$ can be based on straight-line distance to estimate the remaining travel cost.

For example, a route may appear geographically short but may have strong winds and difficult terrain. A* considers these additional costs through $g(n)$. Therefore, it may choose a slightly longer route if that route has lower overall cost and risk.

A* provides a better balance between speed and route quality because it considers both the cost already incurred and the estimated remaining cost.

If the heuristic is admissible, meaning it never overestimates the actual remaining cost, A* can find an optimal solution in a static environment.

III. IMPACT OF HEURISTIC ACCURACY

The accuracy of the heuristic has a significant effect on both Greedy Best-First Search and A* Search.

GREEDY BEST-FIRST SEARCH:

Greedy Best-First Search depends heavily on $h(n)$ because it uses only the heuristic value.

If the heuristic is accurate:

- The algorithm can quickly move toward the destination.
- The search may be efficient.
- Fewer unnecessary nodes may be explored.

If the heuristic is inaccurate:

- The algorithm may choose an unsuitable route.
- It may move toward a blocked or dangerous area.
- It can make poor decisions when weather and terrain conditions are not reflected in the heuristic.
- It does not guarantee an optimal solution.

A* SEARCH:

A* is less dependent on the heuristic because it considers both $g(n)$ and $h(n)$.

If the heuristic is accurate:

- A* can find good routes more quickly.
- The number of unnecessary nodes explored can be reduced.
- The search becomes more efficient.

If the heuristic is weak or inaccurate:

- A* may explore more nodes.
- The search can become slower.
- However, when an admissible heuristic is used in a static environment, A* can still guarantee an optimal solution.

Therefore, Greedy Best-First Search is generally more sensitive to heuristic errors, while A* provides a better balance between actual cost and estimated cost.

IV. IMPACT OF DYNAMIC CHANGES AND BLOCKED PATHS

The flood-affected environment is dynamic. A road or path that was available earlier may suddenly become blocked due to rising water, debris, or changing weather conditions.

In Greedy Best-First Search, if the selected route becomes blocked, the drone must update its search and choose another available route. Since GBFS focuses mainly on the heuristic,

it may repeatedly select routes that appear close to the destination but are actually unavailable or risky.

In A*, dynamic changes can also affect the search. If a path becomes blocked, the previous cost information may no longer represent the current environment. The algorithm must update the search graph and recalculate affected routes. This can increase computation time.

For highly dynamic environments, repeated execution of A* or an incremental search technique can be used. The drone can continuously receive real-time information and re-plan its route whenever a path becomes blocked or the weather changes.

Therefore, dynamic changes reduce the effectiveness of a single static search. Real-time sensing and continuous route re-planning are essential for safe drone operation.

V. RECOMMENDATION OF THE MOST SUITABLE ALGORITHM

A* Search is the most suitable basic search algorithm for this scenario because it considers both the actual cost and the estimated remaining cost.

The main reasons are:

1. BALANCE BETWEEN SPEED AND OPTIMALITY:

A* uses $f(n) = g(n) + h(n)$, allowing the drone to balance actual travel cost with the estimated distance to the destination.

2. CONSIDERATION OF VARIABLE COSTS:

A* can include weather, terrain, battery consumption, and risk in $g(n)$. This makes it more suitable than Greedy Best-First Search for a flood-affected environment.

3. BETTER SAFETY:

The drone can assign higher costs to dangerous areas such as strong winds, flooded regions, or unstable terrain. Therefore, A* can prefer safer routes even if they are slightly longer.

4. OPTIMALITY:

With an appropriate admissible heuristic and a static environment, A* can provide an optimal route. This is an important advantage when delivering emergency medicines.

5. REAL-TIME RE-PLANNING:

When new information about blocked paths is received, the drone can update the search and calculate a new route. For a highly dynamic environment, A* can be combined with incremental or dynamic path-planning methods to improve re-planning efficiency.

COMPARISON OF GREEDY BEST-FIRST SEARCH AND A*

FACTOR	GREEDY BEST-FIRST SEARCH	A* SEARCH
Evaluation Function	$f(n) = h(n)$	$f(n) = g(n) + h(n)$
Main Focus	Estimated distance to goal	Actual cost + estimated cost
Speed	Generally faster	May be slower
Optimality	Not guaranteed	Guaranteed with suitable conditions
Variable Costs	Poorly considered	Can be included in $g(n)$
Safety	May select risky routes	Can consider risk as cost
Heuristic Dependence	Highly dependent	Less dependent
Dynamic Environment	Requires re-planning	Requires re-planning and cost updates
Suitability	Quick approximate decisions	Safer and cost-aware route planning

CONCLUSION

For emergency medicine delivery using AI-powered drones in a flood-affected region, A* Search is the more suitable choice compared with Greedy Best-First Search. Greedy Best-First Search provides faster decisions by focusing only on the estimated distance to the destination, but it may ignore weather, terrain, risk, and actual travel costs. A* provides a better balance by considering both the actual cost and the estimated remaining cost.

However, because the environment is highly dynamic, neither algorithm should rely on a single fixed route. The drone should continuously receive real-time information about blocked paths and weather conditions and re-plan its route when necessary. For practical deployment, A* combined with real-time sensing and dynamic or incremental re-planning would provide a strong balance of speed, optimality, reliability, and safety. Therefore, A* is recommended as the primary route-planning approach for this emergency drone delivery scenario.

2. Scenario: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

- i. Explain how Hill Climbing would work in this scenario and identify its limitations
- ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation
- iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations
- iv. Recommend the best approach and justify your choice based on scalability and adaptability

ANSWER:

SCENARIO: AI-BASED TRAFFIC SIGNAL OPTIMIZATION IN A SMART CITY INTRODUCTION

A smart city implements an AI-based system to optimize traffic signal timings across hundreds of intersections. The main objective is to reduce total vehicle waiting time, fuel consumption, and traffic congestion. The system must continuously adjust traffic signals according to real-time traffic flow. However, the problem has an extremely large search space because each intersection can have different signal timings and coordination patterns. In addition, traffic conditions change dynamically due to accidents, weather, peak hours, and special events. The optimization system may also get stuck in locally optimal solutions. Therefore, an efficient optimization technique is required to find good solutions in a large and changing environment.

FORMULATION AS AN OPTIMIZATION PROBLEM

The traffic signal optimization problem can be formulated as a search and optimization problem.

The system needs to determine the best signal timing configuration for all intersections in the city.

A possible objective function is:

Minimize:

$$F = w_1(W) + w_2(Fc) + w_3(C)$$

where:

W = Total vehicle waiting time

Fc = Total fuel consumption

C = Traffic congestion level

$w_1, w_2,$ and w_3 = Weights representing the importance of each objective

The decision variables can include:

1. Green light duration at each intersection.
2. Red light duration at each intersection.
3. Yellow light duration.
4. Signal phase sequence.
5. Offset between signals at neighboring intersections.
6. Coordination of traffic signals along major roads.

The objective is to find a signal configuration that minimizes waiting time, fuel consumption, and congestion while satisfying constraints such as minimum and maximum green-light

duration, pedestrian crossing requirements, traffic safety rules, and emergency vehicle priority.

WHY TRADITIONAL SEARCH IS INEFFICIENT

Traditional search methods are inefficient for this problem because the number of possible signal configurations becomes extremely large when hundreds of intersections are involved. For example, if each intersection has several possible timing combinations, the total number of possible solutions grows exponentially with the number of intersections. Searching every possible combination would require enormous computational time.

Traditional search methods also face the following problems:

1. **LARGE SEARCH SPACE:**
Hundreds of intersections create a huge number of possible signal timing combinations.
2. **DYNAMIC TRAFFIC CONDITIONS:**
Traffic flow changes continuously, so a solution that is optimal now may become ineffective after a few minutes.
3. **MULTIPLE OBJECTIVES:**
The system must optimize waiting time, fuel consumption, and congestion simultaneously.
4. **LOCAL OPTIMA:**
The search may find a solution that is better than nearby solutions but is not the best solution overall.
5. **REAL-TIME REQUIREMENTS:**
The system must make decisions quickly and continuously update signal timings.

Therefore, traditional exhaustive search is not practical. Intelligent optimization techniques are required to efficiently explore the large search space.

I. HOW HILL CLIMBING WORKS IN THIS SCENARIO

Hill Climbing is a local search optimization technique that starts with an initial solution and continuously moves toward a better neighboring solution.

In the smart city traffic scenario, an initial traffic signal configuration is selected. The algorithm evaluates its performance using the objective function. It then modifies one or more signal timings to create a neighboring solution.

For example:

Initial Solution:

Intersection A → Green = 40 seconds

Intersection B → Green = 50 seconds

Intersection C → Green = 35 seconds

The algorithm changes the timings slightly and evaluates the new configuration.

If the new configuration reduces total waiting time, fuel consumption, or congestion, the algorithm accepts the new solution. This process continues until no neighboring solution provides an improvement.

The basic process is:

1. Generate an initial traffic signal configuration.
2. Calculate the objective function.
3. Generate neighboring signal configurations.
4. Compare the neighboring solutions.
5. Move to the better solution.
6. Repeat until no improvement is found.

LIMITATIONS OF HILL CLIMBING

1. It can get trapped in local optima.
2. It may stop at a plateau where many solutions have similar performance.

3. It can have difficulty moving through ridges.
4. The final solution depends strongly on the initial solution.
5. It does not normally explore distant solutions.
6. It may not adapt well when traffic patterns change suddenly.
7. It may find a locally good traffic signal configuration instead of the globally best configuration.

Therefore, Hill Climbing is simple and computationally efficient, but it may not be sufficient for a large-scale smart city traffic optimization problem.

II. LOCAL MAXIMA, PLATEAUS, AND RIDGES WITH REAL-WORLD INTERPRETATION

Hill Climbing can face several problems during optimization. These problems can be understood using real-world traffic examples.

1. LOCAL MAXIMA

A local maximum is a solution that is better than all nearby solutions but is not the best solution in the entire search space.

REAL-WORLD EXAMPLE:

Suppose the traffic signal system adjusts the timings of five nearby intersections and finds a configuration that significantly reduces congestion in one area. Small changes to the timings make the traffic condition worse, so the algorithm stops.

However, a completely different signal coordination strategy could reduce congestion across the entire city even more. Hill Climbing cannot easily reach this better solution because it would have to temporarily move through worse solutions.

Therefore, the algorithm becomes trapped in a local optimum.

2. PLATEAUS

A plateau occurs when many neighboring solutions have the same or nearly the same objective value.

REAL-WORLD EXAMPLE:

Suppose changing a traffic signal from 45 seconds to 46 seconds or 44 seconds produces almost no difference in traffic waiting time. The algorithm may find many configurations with similar performance.

Since there is no clear improvement, Hill Climbing may stop or move randomly. This can slow down the optimization process.

3. RIDGES

A ridge occurs when the best direction of improvement cannot be reached through a simple single-step movement.

REAL-WORLD EXAMPLE:

Suppose two connected intersections need their green-light durations to increase together. Increasing only one signal may increase congestion, while increasing both signals together would significantly improve traffic flow.

Hill Climbing may not recognize this combined improvement because it evaluates small individual changes. As a result, it may struggle to move along the ridge toward a better solution.

III. HOW SIMULATED ANNEALING AND GENETIC ALGORITHMS OVERCOME THESE LIMITATIONS

A. SIMULATED ANNEALING

Simulated Annealing is inspired by the cooling process used in metallurgy. Unlike Hill Climbing, it can sometimes accept a worse solution in order to escape from a local optimum. In the traffic signal scenario, the algorithm starts with a high temperature. At this stage, it can accept both better and some worse solutions.

As the temperature decreases, the algorithm becomes more selective and increasingly accepts only better solutions.

The general process is:

1. Start with an initial traffic signal configuration.
2. Generate a neighboring solution.
3. Calculate the change in objective value.
4. Accept a better solution.
5. Sometimes accept a worse solution based on a probability.
6. Gradually reduce the temperature.
7. Continue until the system reaches a stable solution.

ADVANTAGES:

1. Can escape local optima.
2. Can explore a large search space.
3. Can handle complex optimization problems.
4. Requires less memory than population-based methods.
5. Useful when traffic conditions change and re-optimization is required.

LIMITATION:

The performance depends on the cooling schedule and parameter selection. If the temperature decreases too quickly, the algorithm may become trapped in a local optimum. If it decreases too slowly, the optimization may take too much time.

B. GENETIC ALGORITHM

A Genetic Algorithm (GA) is inspired by natural evolution. Instead of working with only one solution, it maintains a population of possible traffic signal configurations.

Each solution is represented as a chromosome. The quality of each solution is measured using a fitness function.

For example, a chromosome may represent the signal timings of multiple intersections.

The main steps are:

1. Generate an initial population of signal timing configurations.
2. Calculate the fitness of each solution.
3. Select better solutions as parents.
4. Apply crossover to combine solutions.
5. Apply mutation to introduce new variations.
6. Create a new generation.
7. Repeat the process until a satisfactory solution is obtained.

ADVANTAGES:

1. Searches multiple areas of the solution space simultaneously.
2. Reduces the risk of getting trapped in a single local optimum.
3. Works well with very large and complex search spaces.
4. Can optimize multiple objectives.
5. Mutation provides additional diversity.
6. Can adapt to changing traffic conditions by continuously generating new solutions.

LIMITATION:

Genetic Algorithms may require significant computational resources because they evaluate many solutions in each generation. They also require careful selection of population size, crossover rate, mutation rate, and fitness function.

IV. RECOMMENDATION OF THE BEST APPROACH

For a smart city with hundreds of traffic intersections, a Genetic Algorithm combined with real-time traffic data and local optimization is the most suitable approach.

The main reasons are:

1. **SCALABILITY:**

The traffic signal problem has a very large search space. Genetic Algorithms can evaluate multiple candidate solutions simultaneously and explore different regions of the search space.

2. **AVOIDING LOCAL OPTIMA:**

Unlike Hill Climbing, Genetic Algorithms maintain a population of solutions. Crossover and mutation allow the algorithm to explore new solutions and reduce the possibility of getting trapped in local optima.

3. **MULTI-OBJECTIVE OPTIMIZATION:**

The system must minimize waiting time, fuel consumption, and congestion simultaneously. A Genetic Algorithm can use a multi-objective fitness function to balance these requirements.

4. **ADAPTABILITY:**

Traffic conditions change continuously. New traffic data can be used to update the population and optimize signal timings again. This makes the approach suitable for dynamic smart city environments.

5. **GLOBAL SEARCH:**

Hill Climbing mainly performs local search, while Genetic Algorithms explore multiple areas of the search space. This increases the probability of finding high-quality solutions.

6. **REAL-TIME DECISION MAKING:**

For real-time operation, the Genetic Algorithm can be implemented with suitable population sizes and limited optimization cycles. The system can periodically re-optimize signal timings based on current traffic conditions.

COMPARISON OF OPTIMIZATION METHODS

FACTOR	HILL CLIMBING	SIMULATED ANNEALING	GENETIC ALGORITHM
Search Type	Local Search	Probabilistic Search	Population-Based Search
Local Optima	Easily Trapped	Can Escape	Less Likely to Get Trapped
Large Search Space	Limited	Good	Very Good
Scalability	Moderate	Good	Very Good
Dynamic Environment	Limited	Good	Very Good
Multi-Objective Optimization	Limited	Possible	Highly Suitable
Computational Cost	Low	Moderate	Higher
Adaptability	Low to Moderate	Good	Very Good
Suitable for Smart City	Limited	Suitable	Highly Suitable

CONCLUSION

The optimization of traffic signals across hundreds of intersections is a complex and large-scale problem. The objective is to minimize total waiting time, fuel consumption, and traffic congestion while continuously adapting to changing traffic conditions.

3. Scenario: An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

- i. Explain why this problem requires an online search agent instead of offline search
 - ii. Analyze the challenges of partial observability and dynamic environments
 - iii. Describe how the rover builds and updates its internal model
 - iv. Compare online search with uninformed and informed search strategies
- Justify the most suitable approach considering uncertainty, adaptability, and safety.

Answer:

SCENARIO: AUTONOMOUS MARS ROVER FOR EXPLORATION AND SAMPLE COLLECTION

INTRODUCTION

An autonomous rover is deployed on Mars to explore unknown terrain and collect scientific samples. The rover must navigate safely while avoiding hazards such as craters, rocks, steep slopes, and unstable surfaces. Since the rover has limited sensing capability, it cannot observe the complete environment at once. It also does not have a complete map of the terrain before starting its mission.

The rover must continuously explore its surroundings, collect new information, update its knowledge, and make decisions based on the current situation. Communication with Earth may also have significant delays, making continuous human control impractical. Therefore, the rover requires an intelligent decision-making approach that can operate in an unknown environment and adapt its actions as new information becomes available.

I. WHY THIS PROBLEM REQUIRES AN ONLINE SEARCH AGENT INSTEAD OF OFFLINE SEARCH

An online search agent is suitable for the Mars rover because the rover does not have complete information about the environment before beginning its mission.

In offline search, the agent knows the complete state space, available actions, and environmental conditions before searching for a solution. It can construct a complete plan in advance and then execute that plan.

However, the Mars rover faces an unknown environment. It may not know the location of all craters, rocks, slopes, or safe paths. Therefore, it cannot create a complete route before starting its exploration.

An online search agent follows a different process:

1. The rover observes its current surroundings.
2. It uses available sensor information to understand the environment.
3. It selects an action based on its current knowledge.
4. It moves through the environment.
5. It observes new areas and discovers new information.
6. It updates its internal model.
7. It changes or improves its plan based on the new information.
8. It repeats the process until the mission is completed.

For example, the rover may initially identify a path that appears safe. After moving forward, its sensors may detect a large crater blocking the path. An offline pre-planned

route may fail in this situation. An online search agent can update its knowledge and find an alternative route.

Therefore, online search is required because the rover must make decisions while exploring and cannot depend on a complete map or a fixed plan.

II. CHALLENGES OF PARTIAL OBSERVABILITY AND DYNAMIC ENVIRONMENTS

A. PARTIAL OBSERVABILITY

Partial observability means that the rover cannot observe the entire environment at a given time.

The rover's sensors may have limited range and accuracy. It may only detect objects that are nearby or within its sensor range.

For example, a crater may be hidden behind a rock or may not be detected until the rover moves closer to it.

The main challenges of partial observability include:

1. **INCOMPLETE INFORMATION:**
The rover does not know the complete terrain layout.
2. **HIDDEN HAZARDS:**
Some craters, rocks, or dangerous slopes may remain undetected until the rover approaches them.
3. **SENSOR LIMITATIONS:**
Sensors may have limited range, accuracy, and reliability.
4. **UNCERTAINTY:**
The rover may not be completely certain whether a particular path is safe.
5. **DECISION-MAKING:**
The rover must make decisions using incomplete information.
6. **MAP UNCERTAINTY:**
The rover's map may initially contain large unknown regions.

Therefore, the rover must continuously sense the environment and update its understanding of the terrain.

B. DYNAMIC ENVIRONMENT

Although the Martian environment is relatively static compared with a city, conditions can still change during the mission.

Examples include:

1. Dust storms reducing visibility.
2. Changes in lighting conditions.
3. Loose rocks moving or becoming unstable.
4. Wheel slippage caused by terrain conditions.
5. Changes in energy availability.
6. Unexpected obstacles discovered during exploration.

These changes can affect the safety and cost of movement.

For example, a route that was considered safe may become difficult to traverse due to reduced visibility or unexpected terrain conditions. The rover must therefore continuously monitor its surroundings and adjust its plan.

III. HOW THE ROVER BUILDS AND UPDATES ITS INTERNAL MODEL

The rover creates an internal representation of the environment using sensor data and observations collected during exploration.

The process can be explained as follows:

1. INITIAL STATE

At the beginning, the rover has only limited information about its starting location and nearby surroundings. The rest of the environment is unknown.

2. SENSING THE ENVIRONMENT

The rover uses available sensors such as cameras, LiDAR, and other terrain sensors to observe nearby objects and terrain features.

3. IDENTIFYING OBJECTS AND HAZARDS

The rover analyzes sensor information to identify:

- Rocks
- Craters
- Slopes
- Obstacles
- Safe terrain
- Potential sample collection locations

4. BUILDING A MAP

The rover converts the collected information into an internal map. The map can represent areas as:

- Known safe
- Known dangerous
- Unknown
- Unexplored

For example:

SAFE AREA → Can be traversed

HAZARD AREA → Must be avoided

UNKNOWN AREA → Requires further exploration

5. UPDATING THE MAP

As the rover moves forward, it collects new sensor data. The newly discovered information is added to the internal map.

For example, if the rover discovers a crater in an area previously marked as unknown, it updates that region as dangerous.

6. REPLANNING

When the internal model changes, the rover evaluates whether its current route is still suitable. If the planned path becomes unsafe, it searches for an alternative route.

7. CONTINUOUS LEARNING

The rover continues this cycle:

SENSE → ANALYZE → UPDATE MODEL → PLAN → ACT → SENSE AGAIN

This continuous process allows the rover to operate effectively in an unknown environment.

IV. COMPARISON OF ONLINE SEARCH WITH UNINFORMED AND INFORMED SEARCH STRATEGIES

A. UNINFORMED SEARCH

Uninformed search algorithms, such as Breadth-First Search (BFS) and Depth-First Search (DFS), do not use additional knowledge about how close a state is to the goal. They search using only the problem definition and available actions.

For the Mars rover, uninformed search may explore many unnecessary paths because it does not have information about the quality or safety of different routes.

B. INFORMED SEARCH

Informed search algorithms, such as Greedy Best-First Search and A*, use heuristic information to guide the search.

For example, the rover can use the estimated distance to a sample location as a heuristic.

A* can use:

$$f(n) = g(n) + h(n)$$

where:

$g(n)$ = Actual cost of reaching the current location

$h(n)$ = Estimated cost of reaching the goal

Informed search is generally more efficient than uninformed search when a good heuristic is available.

However, traditional informed search usually assumes that the search space is known sufficiently well. In the Mars rover scenario, the environment is initially unknown and information is revealed gradually.

C. ONLINE SEARCH

Online search is specifically designed for situations where the agent must act while discovering the environment.

The rover does not need to know the complete map before beginning. Instead, it explores, gathers information, updates its internal model, and makes new decisions.

COMPARISON TABLE

FACTOR	UNINFORMED SEARCH	INFORMED SEARCH	ONLINE SEARCH
Environment Knowledge	Requires known problem representation	Requires known problem representation and heuristic	Works with incomplete or unknown information
Heuristic	Not used	Used	Can use available heuristic information
Exploration	May explore many unnecessary states	More directed toward the goal	Explores while acting
Unknown Environment	Not suitable	Limited suitability	Highly suitable
Real-Time Adaptation	Low	Moderate	High
Map Requirement	Usually requires known state space	Usually requires a known search model	Does not require a complete map
Safety in Unknown Terrain	Limited	Depends on available information	Can adapt to newly discovered hazards
Knowledge Update	Limited	Usually based on a fixed model	Continuously updates internal model
Mars Rover Suitability	Low	Moderate	Very High

V. MOST SUITABLE APPROACH FOR THE MARS ROVER

An online search agent is the most suitable approach for the autonomous Mars rover because the environment is unknown, information is incomplete, and the rover must continuously adapt its decisions.

The main reasons are:

1. HANDLING UNCERTAINTY:

The rover cannot observe the entire environment at once. Online search allows it to make decisions using the information currently available and improve those decisions as new information is discovered.

2. ADAPTABILITY:

When the rover detects a new crater or obstacle, it can update its internal model and modify its route. This makes online search more adaptable than a fixed offline plan.

3. SAFETY:

Safety is critical because the rover is operating far from human assistance. Online search allows the rover to react to newly detected hazards instead of blindly following a pre-planned route.

4. UNKNOWN TERRAIN:

The rover does not have a complete map of Mars. Online search allows it to explore unknown regions and gradually build a map.

5. REAL-TIME DECISION MAKING:

The rover must make decisions based on current sensor information. Online search supports the continuous cycle of sensing, planning, and acting.

6. COMBINATION WITH HEURISTICS:

Online search can also use informed strategies when useful. For example, the rover can use distance, terrain difficulty, energy consumption, and hazard risk to select promising paths.

7. MISSION EFFICIENCY:

The rover can balance exploration and goal achievement. It can select routes that are not only short but also safe and energy-efficient.

RECOMMENDATION

The best solution is to use an **ONLINE SEARCH AGENT** combined with **INFORMED SEARCH** and **REAL-TIME SENSOR-BASED PLANNING**.

The rover can use heuristic information to estimate the cost of reaching sample locations while continuously updating its internal map from sensor observations. A cost function can consider:

- Distance
- Terrain difficulty
- Energy consumption
- Hazard risk
- Slope
- Wheel traction
- Scientific importance of sample locations

The rover can then select a route that provides a good balance between exploration, safety, energy efficiency, and mission objectives.

A suitable architecture can be represented as:

SENSORS



ENVIRONMENT OBSERVATION



INTERNAL MODEL / MAP UPDATE



ONLINE SEARCH



HEURISTIC-BASED ROUTE SELECTION



SAFETY CHECK



ACTION / MOVEMENT



NEW OBSERVATION



MODEL UPDATE AND RE-PLANNING

CONCLUSION

The autonomous Mars rover problem requires an online search agent because the rover operates in an unknown environment without a complete map. Unlike offline search, online search allows the rover to explore the environment while making decisions and continuously updating its knowledge.

Partial observability creates challenges because the rover cannot see all hazards in advance, while changing environmental conditions can make previously selected routes unsafe. The rover therefore needs to build an internal model using sensor observations and update it continuously.

Uninformed search methods are less suitable because they do not effectively use available information about the environment. Informed search methods such as A* can improve route selection using heuristic information, but they are more effective when the environment is sufficiently known.

Therefore, the most suitable approach is an online search agent combined with heuristic-based planning, real-time sensing, continuous map updating, and dynamic re-planning. This approach provides the best balance of uncertainty handling, adaptability, safety, energy efficiency, and reliable autonomous exploration, making it highly suitable for a Mars rover operating in unknown terrain.

4. Scenario: A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

- i. Clearly define variables, domains, and constraints with explanation**
- ii. Explain how backtracking search works in this scenario**
- iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency**
- iv. Analyze real-world challenges and justify the best strategy for scalability**

Answer:

SCENARIO: AI-BASED UNIVERSITY EXAM TIMETABLE GENERATION USING CONSTRAINT SATISFACTION

INTRODUCTION

A university is designing an AI-based system to automatically generate exam timetables for its students and courses. The timetable must satisfy multiple constraints, including avoiding overlapping examinations for students, managing a limited number of examination halls, scheduling certain subjects before others, considering faculty availability, and providing special accommodations for students with specific requirements.

As the number of courses, students, faculty members, and examination halls increases, the complexity of timetable generation also increases significantly. Finding a valid timetable manually becomes difficult and time-consuming. Therefore, the problem can be formulated as a Constraint Satisfaction Problem (CSP), where the AI system searches for an assignment of examination times and halls that satisfies all required constraints.

FORMULATION AS A CONSTRAINT SATISFACTION PROBLEM (CSP)

A Constraint Satisfaction Problem consists of three main components:

$CSP = (X, D, C)$

where:

X = Set of variables

D = Set of domains

C = Set of constraints

For the university examination timetable problem, each examination must be assigned a suitable time slot and examination hall while satisfying all constraints.

The main objective is:

"Assign every examination to a valid time slot and examination hall such that all hard constraints are satisfied and the timetable is as efficient and convenient as possible."

I. VARIABLES, DOMAINS, AND CONSTRAINTS

A. VARIABLES

Variables represent the examinations that need to be scheduled.

Suppose the university has the following examinations:

E1 = Data Structures

E2 = Artificial Intelligence

E3 = Computer Architecture

E4 = Database Management Systems

E5 = Mathematics

Each examination is represented as a variable.

For every examination, the system needs to determine:

- 1. Examination time slot.**
- 2. Examination hall.**
- 3. Faculty or invigilator assignment, if required.**

Therefore, a variable can be represented as:

$E_i = (\text{Time Slot}, \text{Hall})$

For example:

$E_1 = (\text{Monday 9:00 AM}, \text{Hall 1})$

$E_2 = (\text{Tuesday 2:00 PM}, \text{Hall 2})$

The complete timetable is the assignment of values to all examination variables.

B. DOMAINS

The domain represents the possible values that can be assigned to each variable.

TIME SLOT DOMAIN:

Suppose the university has the following available examination slots:

$D(\text{Time}) = \{\text{Monday-9AM}, \text{Monday-2PM}, \text{Tuesday-9AM}, \text{Tuesday-2PM}\}$

HALL DOMAIN:

Suppose the available examination halls are:

$D(\text{Hall}) = \{\text{Hall 1}, \text{Hall 2}, \text{Hall 3}\}$

Therefore, each examination can be assigned a combination of a time slot and a hall.

For example:

$D(E_1) = \{(\text{Monday-9AM}, \text{Hall 1}),$

$(\text{Monday-9AM}, \text{Hall 2}),$

$(\text{Monday-2PM}, \text{Hall 1}),$

$(\text{Monday-2PM}, \text{Hall 2}), \dots\}$

The AI system must select one valid combination for every examination.

C. CONSTRAINTS

Constraints define the rules that must be satisfied by the timetable.

1. NO STUDENT EXAM OVERLAP CONSTRAINT

A student cannot attend two examinations scheduled at the same time.

For example, if Student S1 is enrolled in both Data Structures and Artificial Intelligence, these two examinations cannot occur simultaneously.

$E_1.\text{Time} \neq E_2.\text{Time}$

if students are enrolled in both E1 and E2.

This is a hard constraint because violating it makes the timetable invalid.

2. LIMITED EXAM HALL CONSTRAINT

The number of examinations scheduled at the same time cannot exceed the number of available halls.

For example, if the university has only three halls, no more than three examinations can be scheduled simultaneously.

$\text{Number of Exams at Time } T \leq \text{Number of Available Halls}$

3. SUBJECT PRECEDENCE CONSTRAINT

Certain subjects must be completed before other subjects.

For example, if Mathematics must be completed before Advanced Mathematics:

$\text{Mathematics.Time} < \text{Advanced Mathematics.Time}$

This ensures that prerequisite subjects are scheduled earlier.

4. FACULTY AVAILABILITY CONSTRAINT

Faculty members or invigilators may be unavailable during certain periods.

For example, if a faculty member is unavailable on Tuesday afternoon, examinations requiring that faculty member cannot be scheduled during that time.

$\text{Exam.Time} \notin \text{Faculty.UnavailableSlots}$

5. SPECIAL ACCOMMODATION CONSTRAINT

Some students may require special arrangements, such as accessible examination halls, extra time, or specific scheduling requirements.

For example, a student with a special accommodation may require an accessible hall.

$\text{Exam.Hall} \in \text{AccessibleHalls}$

The system must ensure that the student's requirements are satisfied.

6. EXAM HALL CAPACITY CONSTRAINT

The number of students assigned to a hall cannot exceed its capacity.

$\text{Number of Students in Exam} \leq \text{Hall Capacity}$

7. ONE EXAMINATION PER HALL AT A TIME

A hall cannot be assigned to two different examinations at the same time.

If:

$\text{E1.Time} = \text{E2.Time}$

then:

$\text{E1.Hall} \neq \text{E2.Hall}$

8. EXAMINATION AVAILABILITY CONSTRAINT

Each examination must be scheduled within the officially available examination period.

$\text{Exam.Time} \in \text{AvailableTimeSlots}$

9. STUDENT REST CONSTRAINT

To improve student convenience, the system may avoid scheduling examinations too close together.

For example, if possible, a student should not have two examinations in consecutive time slots.

This can be treated as a soft constraint if it is desirable but not mandatory.

II. HOW BACKTRACKING SEARCH WORKS IN THIS SCENARIO

Backtracking Search is a systematic search technique used to solve CSPs. It assigns values to variables one at a time and checks whether the current assignment satisfies all constraints.

If a conflict occurs, the algorithm goes back to the previous variable and tries another value.

The process can be explained as follows:

STEP 1: SELECT A VARIABLE

The algorithm selects an examination that needs to be scheduled.

Example:

$\text{E1} = \text{Data Structures}$

STEP 2: SELECT A VALUE

The algorithm chooses a possible time and hall.

Example:

$\text{E1} = \text{Monday-9AM, Hall 1}$

STEP 3: CHECK CONSTRAINTS

The system checks whether the assignment violates any constraints.

It checks:

- Student conflicts
- Hall availability
- Hall capacity
- Faculty availability
- Special accommodations

- Other scheduling requirements

STEP 4: CONTINUE ASSIGNMENT

If the assignment is valid, the algorithm selects the next examination.

Example:

E2 = Artificial Intelligence

STEP 5: DETECT CONFLICT

Suppose E2 is assigned to Monday-9AM, Hall 1.

If Hall 1 is already occupied by E1, this assignment is invalid.

The algorithm tries another available hall.

STEP 6: BACKTRACK IF NECESSARY

If no valid value is available for E2, the algorithm returns to E1 and changes its assignment.

For example:

Previous:

E1 = Monday-9AM, Hall 1

New:

E1 = Monday-2PM, Hall 2

The algorithm then tries scheduling E2 again.

STEP 7: COMPLETE THE TIMETABLE

The process continues until all examinations are assigned valid time slots and halls.

The basic process is:

SELECT VARIABLE

↓

ASSIGN VALUE

↓

CHECK CONSTRAINTS

↓

IF VALID → CONTINUE

↓

IF INVALID → TRY ANOTHER VALUE

↓

IF NO VALUE WORKS → BACKTRACK

↓

REPEAT UNTIL COMPLETE TIMETABLE

Backtracking guarantees that the system systematically explores possible solutions, but it can become computationally expensive when the number of courses and constraints becomes very large.

III. HOW CONSTRAINT PROPAGATION AND FORWARD CHECKING IMPROVE EFFICIENCY

Constraint propagation improves the efficiency of CSP solving by reducing the possible values in the domains of unassigned variables.

Forward checking is one important constraint propagation technique.

After assigning a value to one variable, the algorithm checks the domains of future variables and removes values that would cause conflicts.

For example:

Suppose:

E1 = Monday-9AM, Hall 1

If E2 and E3 share students with E1, they cannot be scheduled at Monday-9AM.

Therefore, the system removes Monday-9AM from the available time domain of E2 and E3.

Before Forward Checking:

E2 = {Mon-9AM, Mon-2PM, Tue-9AM, Tue-2PM}

After assigning E1:

E2 = {Mon-2PM, Tue-9AM, Tue-2PM}

The invalid value is removed before the algorithm tries it.

ADVANTAGES OF FORWARD CHECKING

1. EARLY CONFLICT DETECTION

Conflicts are detected immediately rather than after a complete timetable is constructed.

2. REDUCED SEARCH SPACE

Invalid values are removed from future domains, reducing the number of possibilities that need to be explored.

3. FEWER BACKTRACKING STEPS

Since many conflicts are detected early, the algorithm does not need to repeatedly return to earlier assignments.

4. IMPROVED PERFORMANCE

The algorithm can find valid timetables faster, especially when there are many constraints.

5. BETTER SCALABILITY

Forward checking becomes increasingly useful as the number of courses and students increases.

Example:

Suppose three examinations share students:

E1, E2, and E3

If:

E1 = Monday-9AM

Then forward checking immediately removes Monday-9AM from the domains of E2 and E3.

This prevents direct scheduling conflicts.

IV. REAL-WORLD CHALLENGES IN EXAM TIMETABLE GENERATION

A real university timetable system faces many practical challenges.

1. LARGE NUMBER OF COURSES

Universities may offer hundreds or thousands of courses. The number of possible timetable combinations grows rapidly.

2. LARGE NUMBER OF STUDENTS

Students may enroll in multiple courses. Identifying all student conflicts can be computationally expensive.

3. LIMITED EXAM HALLS

The number and capacity of examination halls are limited, creating resource constraints.

4. FACULTY AVAILABILITY

Faculty members may have teaching schedules, leave, meetings, or other responsibilities.

5. SPECIAL ACCOMMODATIONS

Students with accessibility requirements may need specific halls, additional time, or special arrangements.

6. SUBJECT DEPENDENCIES

Some subjects may need to be scheduled before others due to prerequisite relationships.

7. LAST-MINUTE CHANGES

A faculty member may become unavailable, a hall may become unavailable, or a course may be added or removed.

8. MULTIPLE OBJECTIVES

The system should not only produce a valid timetable but also minimize student inconvenience.

For example, it should try to:

- Reduce consecutive examinations.
- Avoid excessive gaps between exams.
- Balance examination loads.
- Reduce student travel between halls.
- Improve hall utilization.

9. DYNAMIC CONDITIONS

The timetable may need to be modified when unexpected events occur, such as emergencies, holidays, or changes in university schedules.

V. BEST STRATEGY FOR SCALABILITY

For a large university, the best approach is a combination of:

CSP + BACKTRACKING SEARCH + CONSTRAINT PROPAGATION + HEURISTICS

The system should use backtracking as the basic search mechanism, while constraint propagation reduces the search space.

The following strategies can further improve scalability:

1. MINIMUM REMAINING VALUES (MRV)

Select the examination with the smallest remaining domain first.

This means the most constrained examination is scheduled first.

2. DEGREE HEURISTIC

If two examinations have similar domain sizes, select the examination that is involved in the largest number of constraints.

For example, an examination shared by many students should be scheduled earlier.

3. LEAST CONSTRAINING VALUE (LCV)

Select the value that leaves the maximum number of options available for other examinations.

4. FORWARD CHECKING

Remove invalid values from the domains of future variables after each assignment.

5. ARC CONSISTENCY

Use techniques such as Arc Consistency to remove values that cannot participate in any valid solution.

6. SOFT CONSTRAINT OPTIMIZATION

After satisfying all hard constraints, optimize soft constraints such as student convenience, balanced schedules, and reduced gaps.

7. HYBRID APPROACH

For very large universities, CSP techniques can be combined with optimization algorithms such as Genetic Algorithms or Local Search to improve the quality and scalability of the timetable.

RECOMMENDED STRATEGY

The recommended architecture is:
COURSE AND STUDENT DATA



DEFINE VARIABLES AND DOMAINS



DEFINE HARD AND SOFT CONSTRAINTS



MRV VARIABLE SELECTION



BACKTRACKING SEARCH



FORWARD CHECKING / CONSTRAINT PROPAGATION



CHECK HARD CONSTRAINTS



OPTIMIZE SOFT CONSTRAINTS



GENERATE FINAL EXAM TIMETABLE

COMPARISON OF CSP STRATEGIES

STRATEGY	ADVANTAGES	LIMITATIONS	SUITABILITY
Basic Backtracking	Simple and complete search	Can be slow for large problems	Small CSPs
Backtracking + MRV	Selects difficult variables first	Requires additional calculations	Good
Forward Checking	Detects conflicts early	May still require backtracking	Very Good
Arc Consistency	Strong domain reduction	Higher computational overhead	Large CSPs
CSP + Heuristics	Efficient variable and value selection	Requires good heuristic design	Highly Suitable
Hybrid CSP + Optimization	Handles complex large-scale problems	More complex implementation	Best for very large universities

CONCLUSION

The university examination timetable generation problem can be effectively modeled as a Constraint Satisfaction Problem (CSP). The examinations are represented as variables, possible time slots and halls form the domains, and rules such as no student overlap, limited hall capacity, faculty availability, subject precedence, and special accommodations form the constraints.

Backtracking Search provides a systematic method for finding a valid timetable by assigning values one at a time and returning to previous decisions when conflicts occur. However, basic backtracking can become inefficient as the number of courses and students increases.

5. Scenario: Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning)

A gaming company is developing an AI opponent for a real-time strategy game. The

AI must compete against human players, Make decisions within strict time limits, Evaluate

complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

- i. Explain how the Minimax algorithm models this problem
- ii. Analyze the computational challenges of large game trees
- iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning
- iv. Design an evaluation function and justify the factors considered
- vi. Recommend a strategy for balancing optimality and real-time performance

ANSWER:

SCENARIO: STRATEGIC GAME AI FOR REAL-TIME DECISION MAKING USING MINIMAX AND ALPHA-BETA PRUNING

INTRODUCTION

A gaming company is developing an Artificial Intelligence (AI) opponent for a real-time strategy game. The AI must compete against human players and make intelligent decisions within strict time limits. During the game, the AI has to evaluate complex game states, choose among many possible moves, and predict the possible responses of the opponent.

The decision tree of a real-time strategy game can become extremely large because each game state may have many possible actions, and each action can lead to several future states. Searching the entire game tree is computationally expensive and may not be possible within the available time. Therefore, game-playing algorithms such as Minimax and Alpha-Beta Pruning are used to make intelligent decisions efficiently.

I. HOW THE MINIMAX ALGORITHM MODELS THIS PROBLEM

The Minimax algorithm is a decision-making algorithm commonly used in two-player, competitive games. It assumes that both players are intelligent and that each player tries to maximize their own chances of winning.

In this scenario:

MAX = AI Player

MIN = Human Opponent

The AI attempts to maximize its score, while the human opponent attempts to minimize the AI's score.

The game can be represented as a tree:

ROOT STATE

↓

AI POSSIBLE MOVES (MAX)

↓

OPPONENT POSSIBLE MOVES (MIN)

↓

AI POSSIBLE RESPONSES (MAX)

↓

TERMINAL OR DEPTH-LIMITED STATES



EVALUATION FUNCTION

The Minimax algorithm explores possible moves and their consequences. It assumes that the opponent will always choose the move that is most beneficial to them.

The basic process is:

1. The AI identifies all possible legal moves.
2. Each move creates a new game state.
3. The opponent's possible responses are considered.
4. The AI's future responses are considered.
5. The process continues until a terminal state or a predefined search depth is reached.
6. The evaluation function assigns a score to each resulting state.
7. At MAX nodes, the highest score is selected.
8. At MIN nodes, the lowest score is selected.
9. The AI chooses the move that provides the best outcome assuming optimal opponent play.

The Minimax decision rule can be represented as:

MAX chooses:

$\max(\text{value of child states})$

MIN chooses:

$\min(\text{value of child states})$

For example, suppose the AI has three possible moves:

Move A $\rightarrow$ Score = 5

Move B $\rightarrow$ Score = 8

Move C $\rightarrow$ Score = 3

If the AI can directly choose among these outcomes, it selects Move B because 8 is the highest score.

However, if the opponent can respond to each move, the AI must consider the opponent's best response before making the final decision.

Therefore, Minimax models the game as a competitive decision-making problem where each player attempts to optimize their own outcome.

II. COMPUTATIONAL CHALLENGES OF LARGE GAME TREES

The major challenge in real-time strategy games is the enormous size of the game tree.

If:

b = Average branching factor

d = Search depth

Then the approximate number of nodes explored by Minimax is:

$O(b^d)$

where:

b = Number of possible moves at each state

d = Number of levels or moves explored

For example, if a game has:

$b = 10$ possible moves

$d = 6$ levels

Then approximately:

$10^6 = 1,000,000$

game states may need to be evaluated.

If the branching factor or search depth increases, the number of states grows exponentially.

The major computational challenges are:

1. LARGE BRANCHING FACTOR

Real-time strategy games may have many possible actions at every moment.

2. DEEP GAME TREES

The AI may need to predict several moves into the future.

3. REAL-TIME CONSTRAINTS

The AI must make decisions quickly. It cannot spend several minutes searching for the best move.

4. COMPLEX GAME STATES

A state may include:

- Player resources
- Unit positions
- Health
- Buildings
- Technology
- Military strength
- Map control
- Enemy threats

5. MULTIPLE OPPONENT RESPONSES

The opponent may respond in many different ways, creating a large number of possible future states.

6. DYNAMIC GAME ENVIRONMENT

Game states change continuously as players move units, collect resources, build structures, and attack opponents.

7. COMPUTATIONAL COST

Searching every possible future state requires significant CPU and memory resources. Because of these challenges, a complete Minimax search is often impractical for real-time strategy games.

III. HOW ALPHA-BETA PRUNING IMPROVES EFFICIENCY

Alpha-Beta Pruning is an optimization technique used with Minimax. It reduces the number of game states that need to be evaluated without changing the final decision made by Minimax.

It works by eliminating branches of the game tree that cannot possibly influence the final decision.

Alpha and Beta represent bounds:

Alpha (α):

The best value that MAX can guarantee so far.

Beta (β):

The best value that MIN can guarantee so far.

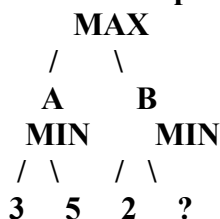
The pruning condition is:

If $\alpha \geq \beta$

Then the remaining branch can be pruned because it cannot affect the final decision.

HOW ALPHA-BETA PRUNING WORKS

Consider a simplified tree:



First, the algorithm evaluates branch A.

MIN chooses:

$\min(3, 5) = 3$

Therefore, MAX currently has a value of 3.

Now the algorithm examines branch B.

The first child of B gives:

2

Since B is a MIN node, it can already guarantee a value of 2 or less.

MAX already has a better option with value 3.

Therefore, MAX will never choose B if B can produce a result of 2 or lower.

The remaining children of B do not need to be evaluated.

They are PRUNED.

This saves computational time while producing the same result as full Minimax.

ADVANTAGES OF ALPHA-BETA PRUNING

1. REDUCES SEARCH SPACE

Many unnecessary branches are eliminated.

2. IMPROVES SPEED

The AI can search deeper within the same time limit.

3. PRESERVES MINIMAX RESULT

With correct implementation, pruning does not change the final decision.

4. SUPPORTS REAL-TIME DECISION MAKING

The AI can make decisions faster.

5. ALLOWS DEEPER SEARCH

Because fewer nodes are evaluated, the AI can search additional levels of the game tree.

6. IMPROVES RESOURCE UTILIZATION

CPU and memory resources are used more efficiently.

BEST AND WORST-CASE PERFORMANCE

Basic Minimax:

$O(b^d)$

Alpha-Beta Pruning worst case:

$O(b^d)$

Alpha-Beta Pruning best case with excellent move ordering:

$O(b^{(d/2)})$

This means that good move ordering can potentially allow the AI to search approximately twice as deep for the same computational effort.

Therefore, Alpha-Beta Pruning is highly effective for real-time game AI.

IV. DESIGN OF AN EVALUATION FUNCTION

In a real-time strategy game, it is usually impossible to search until the actual end of the game because the game tree is too large.

Therefore, the AI stops searching at a fixed depth or when the time limit is reached. An evaluation function is then used to estimate how good the current game state is.

A possible evaluation function is:

Evaluation(S) =

$w_1(\text{Material Advantage})$

- $w_2(\text{Resource Advantage})$**
- $w_3(\text{Territory Control})$**
- $w_4(\text{Unit Health})$**
- $w_5(\text{Strategic Position})$**
- $w_6(\text{Technology Level})$**
- $w_7(\text{Threat Level})$**

where:

S = Current game state

w1, w2, ..., w7 = Weights representing the importance of each factor

The following factors can be considered:

1. MATERIAL ADVANTAGE

Measures the number and strength of military units.

For example:

AI Units - Opponent Units

A greater number of strong units generally indicates a better position.

2. RESOURCE ADVANTAGE

Measures resources such as:

- **Gold**
- **Food**
- **Energy**
- **Minerals**

Having more resources allows the AI to create more units and buildings.

3. TERRITORY CONTROL

Measures how much strategically valuable territory is controlled by the AI.

Controlling important locations can provide tactical advantages.

4. UNIT HEALTH

Measures the health and survival probability of the AI's units.

A large army with low health may be less valuable than a smaller army with high health.

5. STRATEGIC POSITION

Considers whether units are positioned advantageously.

Examples include:

- **Defensive positions**
- **High-ground locations**
- **Protected bases**
- **Important resource areas**

6. TECHNOLOGY LEVEL

Measures the technological development of the AI compared with the opponent.

Advanced technology can provide stronger units and better capabilities.

7. THREAT LEVEL

Measures the immediate danger from the opponent.

For example, an enemy attack near the AI's main base should receive a high threat value.

8. VICTORY CONDITIONS

If the game state represents a winning position, the evaluation function should assign a very high positive value.

If it represents a losing position, it should assign a very high negative value.

A simplified evaluation function can be:

E(S) =

5M + 3R + 4T + 2H + 3P + 2Tech - 5Threat

The weights should be adjusted according to the game design and tested through simulations.

The evaluation function is important because it allows the AI to estimate the quality of a game state without searching until the actual end of the game.

V. RECOMMENDED STRATEGY FOR BALANCING OPTIMALITY AND REAL-TIME PERFORMANCE

The best approach for a real-time strategy game is:

ITERATIVE DEEPENING MINIMAX

+

ALPHA-BETA PRUNING

+

GOOD MOVE ORDERING

+

EFFICIENT EVALUATION FUNCTION

+

TIME LIMIT

This combined approach provides a good balance between decision quality and real-time performance.

1. ITERATIVE DEEPENING

The AI first searches to a shallow depth.

For example:

Depth 1 → Make a quick decision

Depth 2 → Improve the decision

Depth 3 → Search deeper

Depth 4 → Search deeper if time is available

The AI continues increasing the depth until the available time is almost exhausted.

The best result from the deepest completed search is selected.

This ensures that the AI always has a valid decision even if the time limit is reached suddenly.

2. ALPHA-BETA PRUNING

Alpha-Beta Pruning removes branches that cannot affect the final decision. This allows the AI to explore deeper game states within the same time limit.

3. MOVE ORDERING

The AI should evaluate promising moves first.

For example, it can prioritize:

- Winning moves
- Defensive moves
- Capturing valuable enemy units
- Blocking dangerous attacks

Good move ordering increases the effectiveness of Alpha-Beta Pruning.

4. TRANSPOSITION TABLES

The AI can store previously evaluated game states. If the same state is reached through different move sequences, the stored result can be reused instead of recalculating it.

5. TIME-LIMITED SEARCH

The AI should have a strict computation time limit. This prevents the game from becoming unresponsive.

6. EFFICIENT EVALUATION FUNCTION

The evaluation function should be fast enough to calculate thousands or millions of game states without causing delays.

7. DOMAIN-SPECIFIC HEURISTICS

The AI can use game-specific knowledge to prioritize important moves and avoid wasting computation on obviously poor actions.

RECOMMENDED ARCHITECTURE

REAL-TIME GAME STATE



GENERATE POSSIBLE MOVES



MOVE ORDERING



ITERATIVE DEEPENING MINIMAX



ALPHA-BETA PRUNING



EVALUATION FUNCTION



SELECT BEST MOVE



EXECUTE ACTION



UPDATE GAME STATE



REPEAT

COMPARISON OF GAME AI STRATEGIES

FACTOR	MINIMAX	ALPHA-BETA PRUNING	ITERATIVE DEEPENING + ALPHA-BETA
Search Quality	High	High	High
Search Speed	Slow	Faster	Very Fast in practice
Large Game Trees	Difficult	Better	Highly Suitable
Real-Time Performance	Limited	Good	Excellent
Memory Usage	Moderate	Moderate	Moderate
Local Decision Quality	High	High	High
Supports Time Limits	Limited	Good	Excellent
Deeper Search	Expensive	Possible	Efficient
Recommended for RTS	Limited	Suitable	Best Choice

CONCLUSION

The development of an AI opponent for a real-time strategy game is a complex decision-making problem because the game contains a very large decision tree, multiple possible actions, complex game states, and strict time constraints.

The Minimax algorithm models the game as a competitive search problem where the AI attempts to maximize its score while assuming that the human opponent will minimize

it. However, the computational complexity of Minimax, $O(b^d)$, makes complete game-tree exploration impractical for large real-time strategy games.

Alpha-Beta Pruning improves Minimax by eliminating branches that cannot influence the final decision. This significantly reduces the number of states that need to be evaluated and allows the AI to search deeper within the available time.